

Terminal Feature Reporting

Version 1.0 — Draft

Terminal emulators need a flexible way to report their capabilities to applications that communicate with them. The existing mechanism of `$TERM` is too slow to change because `terminfo` updates happen too infrequently in practice, and individual users may have no control.

This proposed standard defines a collection of values that terminals will communicate to apps. It will be exposed through `TERM_FEATURES` and also a to-be-determined control sequence.

Each value has an associated data type. Booleans are represented by a single bit where 1 means it is supported and 0 means it is not supported. Where features take an integer value, the number of bits and signedness will be specified. Signed values will use twos complement.

Control sequences will use some notational shortcuts:

- Byte literals will be given in hex notation or as a character or string literal. For example, `0x41` and `A` are equivalent.
- A series of characters will be written in double quotes, like `"s0"`.
- `ESC` is a synonym for `0x1b`.
- `CSI` is a synonym for `ESC [`.
- `BEL` is a synonym for `0x7`.
- `OSC` is a synonym for `ESC ]`.
- `ST` is a synonym for either `ESC \` or `BEL`.
- `DECSET {code}` is a synonym for `CSI ? {code} h`.
- `DECRST {code}` is a synonym for `CSI ? {code} l`.
- `SGR {code}` indicates that `{code}` is allowed as one of the semicolon delimited arguments to `CSI <arguments> m`.
- Values in `{braces}` are variables.
- Spaces are insignificant.
- Values in `<angle brackets>` will be described in the text.

Some features can be partially or fully validated by [escctest](#). This will be noted in each feature.

Table Stakes

Terminal emulators supporting this protocol must, at a minimum, support the features in this section as specified. Is it worth saying “must support VT100/VT220/ECMA-48 emulation” just to be completely clear, or do those standards include too much legacy cruft that nobody bothers to implement (hi, `stterm`)?

Parsing

SGR, DECSET, and DECRST with multiple parameters must be supported.

Unrecognized but well-formed CSI, OSC, and DCS codes must be properly parsed and ignored.

TODO: Write a test suite of codes that should be ignored.

256 Colors

These control sequences change the foreground and background colors:

- CSI 38 ; 5 ; {fg} m
 - Sets the foreground color to {fg}.
- CSI 48 ; 5 ; {bg} m
 - Sets the background color to {bg}.

The {fg} and {bg} values are interpreted as an index into a color table that is initialized with the following values:

- 0-7: Normal-intensity ANSI colors
- 8-15: Bold/bright intensity ANSI colors
- 16-255: See this table for sRGB values: <https://pastebin.com/raw/nFJKyqDY>

esctest

Use the following tests to validate that the control sequences are parsed correctly and persisted:

- `SGRTests.test_SGR_256Color_Foreground`
- `SGRTests.test_SGR_256Color_Background`

Features

This section lists features. Each feature has a canonical name, which is the section's title.

24BIT

Takes an integer value from:

- 0: No support for 24-bit color
- 1: Supports the compatibility control sequences
- 2: Supports the full control sequences
- 3: Supports both compatibility and full control sequences

The compatibility control sequences are:

- CSI 38 ; 2 ; {red} ; {green} ; {blue} m
 - Sets the foreground color using RGB values where {red}, {green}, and {blue} are integers in [0,255].
- CSI 48 ; 2 ; {red} ; {green} ; {blue} m
 - Sets the background color using RGB values where {red}, {green}, and {blue} are integers in [0,255].

The colorspace is sRGB.

The full control sequences are:

- CSI 38 : 2 : {colorspace} : {red} : {green} : {blue} <more subparameters> m
- CSI 48 : 2 : {colorspace} : {red} : {green} : {blue} <more subparameters> m

The <more subparameters>, if present, should be ignored. This makes it possible to evolve this control sequence in the future.

Terminals should ignore {colorspace} and use sRGB. {red}, {green}, and {blue} take values in 0 through 255 inclusive.

Application Notes

When possible, applications should prefer 24BIT_FULL to 24BIT_COMPAT so that the 24BIT_COMPAT form can eventually be removed.

Because this syntax uses colons to delimit its parameters, other SGR codes could come before the 38/48 or after the last subparameter, as usual in SGR. Applications should leave {colorspace} empty.

escstest

Use the following tests to validate that the compatibility control sequences are parsed correctly and persisted:

- `SGRTests.test_SGR_24BitCompat_Foreground`
- `SGRTests.test_SGR_24BitCompat_Background`

Use the following tests to validate that the full control sequences are parsed correctly and persisted:

- `SGRTests.test_SGR_24BitITU_Foreground`
- `SGRTests.test_SGR_24BitITU_Background`

CLIPBOARD_WRITABLE

Boolean. Indicates if the terminal allows saving to the clipboard using this sequence:

- `OSC 52 ; {clipboard} ; {base64-data}`

Where {clipboard} is either empty or “s0” and {base64-data} is base-64 encoded bytes to put on the clipboard. The character encoding will be UTF-8, both for the encoded and decoded payload.

Terminals should not advertise this feature if the user has disabled it.

Test File

A file with the following bytes should cause “Hello world!” to be copied to the clipboard:

```
1B 5D 35 32 3B 73 30 3B 53 47 56 73 62 47 38 67 64 32 39 79 62 47 51 68 07
0A
```

DECSLRM

Boolean. Indicates if the terminal supports setting the left-right margins. The relevant control sequences are:

- `DECSET 69`
 - Enables left-right margins.

- CSI {left} ; {right} ; s
 - Sets the left-right margins.
- DECRST 69
 - Disables left-right margins.

esctest

Use the following tests to validate support for DECSLRM. If a test depends on a feature the terminal emulator does not support (such as origin mode) then it can be considered optional.

- BSTests.test_BS_ReverseWrapWithLeftRight
- BSTests.test_BS_ReversewrapFromLeftEdgeToRightMargin
- BSTests.test_BS_StopsAtLeftMargin
- BSTests.test_BS_MovesLeftWhenLeftOfLeftMargin
- CBTTests.test_CBT_IgnoresRegion
- CHATests.test_CHA_IgnoresScrollRegion
- CHTTests.test_CHT_IgnoresScrollingRegion
- CNLTests.test_CNL_StopsAtBottomLineWhenBegunBelowScrollRegion
- CNLTests.test_CNL_StopsAtBottomMarginInScrollRegion
- CPLTests.test_CPL_StopsAtTopLineWhenBegunAboveScrollRegion
- CPLTests.test_CPL_StopsAtTopMarginInScrollRegion
- CRTTests.test_CR_MovesToLeftMarginWhenRightOfLeftMargin
- CRTTests.test_CR_MovesToLeftOfScreenWhenLeftOfLeftMargin
- CRTTests.test_CR_StaysPutWhenAtLeftMargin
- CRTTests.test_CR_MovesToLeftMarginWhenLeftOfLeftMarginInOriginMode
- CUBTests.test_CUB_StopsAtLeftEdgeWhenBegunLeftOfScrollRegion
- CUBTests.test_CUB_StopsAtLeftMarginInScrollRegion
- CUFTests.test_CUF_StopsAtRightEdgeWhenBegunRightOfScrollRegion
- CUFTests.test_CUF_StopsAtRightMarginInScrollRegion
- DCHTests.test_DCH_RespectsMargins
- DCHTests.test_DCH_DeleteAllWithMargins
- DECALNTests.test_DECALN_ClearsMargins
- DECBITests.test_DECBI_Scrolls
- DECBITests.test_DECBI_LeftOfMargin
- DECCRATests.test_DECCRA_ignoresMargins
- DECDCTests.test_DECDC_CursorWithinTopBottom
- DECDCTests.test_DECDC_IsNoOpWhenCursorBeginsOutsideScrollRegion
- DECDCTests.test_DECDC_DeleteAllWithLeftRightMargins
- DECFITests.test_DECFI_Scrolls
- DECFITests.test_DECFI_RightOfMargin
- DECICTests.test_DECIC_CursorWithinTopBottom

- DECICTests.test_DECIC_IsNoOpWhenCursorBeginsOutsideScrollRegion
- DECICTests.test_DECIC_ScrollOffRightMarginInScrollRegion
- DECRQSSTests.test_DECRQSS_DECSLRM
- DECSCLTests.test_DSCSCL_Level3_SupportsDECRQMDoesntSupportDECSLRM
- DECSCLTests.test_DECSCL_Level4_SupportsDECSLRMDoesntSupportDECNCMS
- DECSSETests.test_DECSSET_0_WithScrollRegion
- DECSSETests.test_DECSSET_1_WithScrollRegion
- DECSSETests.test_DECSSET_2_WithScrollRegion
- DECSSETests.test_DECSSET_0_WithScrollRegion_Protection
- DECSSETests.test_DECSSET_1_WithScrollRegion_Protection
- DECSSETests.test_DECSSET_2_WithScrollRegion_Protection
- DECSELTTests.test_DECSEL_IgnoresScrollRegion
- DECSELTTests.test_DECSEL_IgnoresScrollRegion_Protection
- DECSETTests.test_DECSET_DECLRMM
- DECSETTests.test_DECSET_DECLRMM_MarginsResetByDECSTR
- DECSETTests.test_DECSET_DECLRMM_ModeNotResetByDECSTR
- DECSTRTests.test_DECSTR_DECLRMM
- DLTests.test_DL_InLeftRightScrollRegion
- DLTests.test_DL_OutsideLeftRightScrollRegion
- DLTests.test_DL_InLeftRightAndTopBottomScrollRegion
- DLTests.test_DL_ClearOutLeftRightAndTopBottomScrollRegion
- DLTests.test_DL_OutsideLeftRightAndTopBottomScrollRegion
- ECHTests.test_ECH_IgnoresScrollRegion
- ECHTests.test_ECH_OutsideScrollRegion
- EDTTests.test_ED_0_WithScrollRegion
- EDTTests.test_ED_1_WithScrollRegion
- EDTTests.test_ED_2_WithScrollRegion
- ELTests.test_EL_IgnoresScrollRegion
- FFTests.test_FF_MovesDoesNotScrollOutsideLeftRight
- FillRectangleTests.fillRectangle_ignoresMargins
- ICHTests.test_ICH_IsNoOpWhenCursorBeginsOutsideScrollRegion
- ICHTests.test_ICH_ScrollOffRightMarginInScrollRegion
- ILTests.test_IL_RespectsScrollRegion
- ILTests.test_IL_RespectsScrollRegion_Over
- ILTests.test_IL_AboveScrollRegion
- INDTests.test_IND_MovesDoesNotScrollOutsideLeftRight
- LFTTests.test_LF_MovesDoesNotScrollOutsideLeftRight
- NELTests.test_NEL_MovesDoesNotScrollOutsideLeftRight
- REPTests.test_REP_RespectsLeftRightMargins
- RITests.test_RI_MovesDoesNotScrollOutsideLeftRight
- RISTests.test_RIS_RemoveMargins
- SaveRestoreCursorTests.test_SaveRestoreCursor_ResetsOriginMode
- SaveRestoreCursorTests.test_SaveRestoreCursor_WorksInLRM

- `SDTests.test_SD_RespectsLeftRightScrollRegion`
- `SDTests.test_SD_OutsideLeftRightScrollRegion`
- `SDTests.test_SD_LeftRightAndTopBottomScrollRegion`
- `SDTests.test_SD_BigScrollLeftRightAndTopBottomScrollRegion`
- `SMTTests.test_SM_IRM_TruncatesAtRightMargin`
- `SUTests.test_SU_RespectsLeftRightScrollRegion`
- `SUTests.test_SU_OutsideLeftRightScrollRegion`
- `SUTests.test_SU_LeftRightAndTopBottomScrollRegion`
- `SUTests.test_SU_BigScrollLeftRightAndTopBottomScrollRegion`
- `VTTests.test_VT_MovesDoesNotScrollOutsideLeftRight`

MOUSE

Unsigned integer. The value is computed by bitwise ORing the subfeature flags that the terminal emulator supports and are currently available. The subfeature flags are:

Motion reporting	1
Left mouse button	2
Right mouse button	4
Middle mouse button	8
Scroll wheel	16
Shift modifier	32
Meta modifier	64
Control modifier	128

A value of 0 indicates no support for mouse reporting.

In order to advertise MOUSE, the terminal must support these control sequences that enable or disable various mouse reporting modes:

- DECSET 1000
- DECRST 1000
- DECSET 1002
- DECRST 1002
- DECSET 1003
- DECRST 1003
- DECSET 1006
- DECRST 1006

Reporting Formats

Traditionally, terminals have supported a number of formats to report mouse events like button clicks, movement, and drags. The older modes had compatibility problems with large terminals and UTF-8. For that reason, terminals advertising support for this feature must implement DECSET 1006, which is safe to use. The older reporting formats may be supported as well, but this document is not concerned with them. See the section **DECSET 1006** for details about how to format a mouse report.

DECSET 1000: Normal tracking mode.

Normal tracking mode sends a report on button press and release.

DECSET 1002: Button event tracking.

Like Normal tracking mode, but includes movements of the mouse while a button is pressed. Motion events are reported only when the mouse moves to another cell.

DECSET 1003: Any event tracking.

Like Button event tracking mode, but includes movements of the mouse while no button is pressed.

DECSET 1006: SGR extended coordinates mode.

Terminals usually use an outmoded and problematic encoding of mouse state in their default configuration. DECSET 1006 enables the modern reporting sequence, as described below.

- Normally: CSI < <params> M
- When reporting a button was released: CSI < <params> m

Where <params> is defined as three numbers represented in decimal using ASCII digits (e.g., “123”), separated by semicolons. The values are:

1. The button/modifiers value, described below.
2. The X coordinate, where 1 is the leftmost column and increasing to the right.
3. The Y coordinate, where 1 is the topmost row and increasing as you move down.

The button/modifiers value is calculated by bitwise ORing values from this table.

Value	Meaning	Modes
0	Left mouse button pressed.	All
1	<i>When 64 is included:</i> When present, scroll wheel moved up. Otherwise, scroll wheel moved down. <i>Otherwise:</i> Right mouse button pressed.	All
2	Other (typically middle, but platform-defined) button pressed.	All
3	<i>When 32 is included:</i> No button pressed. <i>Otherwise:</i> Never used.	1003
4	Shift modifier pressed.	All
8	Meta (or platform-defined equivalent, typically Alt or Option) modifier pressed.	All
16	Control modifier pressed.	All
32	Mouse moved.	All
64	Use scroll wheel interpretation for 1.	All

The behavior when multiple buttons are pressed at the same time is platform-defined.

Because of the limitations of the platform that a terminal emulator operates in, some combinations of mouse buttons and modifiers may not be supported. For example, if the platform lacks a modifier or reserves it so that it cannot be used for mouse reporting, the terminal emulator should not advertise that it supports it. If the mouse lacks a middle button and there's no suitable way to emulate it, the terminal emulator should not advertise that it can report the middle button.

DECSET 1007: Alternate scroll

When enabled and when the terminal is in alternate screen mode, the scroll wheel causes the terminal emulator to send cursor up and cursor down keys.

Precedence

DECSET 1000, 1002, and 1003 take precedence over DECSET 1007 if both 1007 and one of the other modes is enabled at the same time.

If more than one of 1000, 1002, and 1003 are on, the most recently enabled mode takes precedence and automatically disables the previous mode. For example, this sequence disables mouse reporting: DECSET 1000, DECSET 1003, DECRST 1003.

Application Notes

Applications are advised to always set 1006 along with 1000, 1002, or 1003. When exited or suspended, applications should DECSET 1006 along with whichever of 1000, 1002, or 1003 they enabled.

Applications should ignore any subfeature flags they do not recognize.

Tests

https://raw.githubusercontent.com/gnachman/iTerm2/master/tests/mouse_test.sh

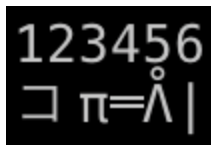
UNICODE_BASIC

Boolean. Indicates if UTF-8 is accepted and basic Unicode features are implemented.

In order to advertise this feature, a terminal emulator should accept [this test file](#) shown below hex-encoded:

```
31 32 33 34 35 36 0D 0A E3 82 B3 CF 80 E2 95 90 CE 9B CC 8A 7C
```

And produce output substantially similar to this:



```
123456
⌋ π=λ |
```

Unicode is big and complex, and it is likely impossible for terminal emulators to conform to all of it and also respect the traditional grid of cells. This feature is meant to set a very low bar for Unicode support, and specifically does not include:

- BiDi
- Scripts that require shaping, like Korean, Thai, or Indic scripts.
- [TODO: a lot more]

Application Notes

Applications generally use `wcwidth` or `wcswidth` to infer where the cursor will be positioned after printing text. The terminal emulator must be in exact agreement with the application or visual artifacts will result.

Terminals' behavior when a control sequence occurs within the bytes of a UTF-8 encoded codepoint, or between the codepoints that represent a grapheme cluster (such as between a base character and a combining mark) is not specified. Applications are advised not to do that.

Applications may expect to see inconsistent behavior between terminal emulators with Wide characters when they appear in unexpected places, such as in the last column, or when a Narrow character overwrites the second cell of a Wide character. This feature does not attempt to address these issues.

DECSCUSR

Boolean. Indicates if the terminal emulator supports DECSCUSR with parameters 0 through 4.

- `CSI {mode} 0x20 q`

This will change the cursor shape depending on mode:

- 1: Blinking block
- 2: Steady block
- 3: Blinking underline
- 4: Steady underline

Terminals may choose not to make the cursor blink at their discretion, since some people find it annoying.

DECSCUSR_BAR

Boolean. Indicates if the terminal emulator supports DECSCUSR with parameters 5 and 6 to change the cursor into a vertical bar.

- `CSI {mode} 0x20 q`

The mode values are:

- 5: Blinking bar
- 6: Steady bar

As with DECSCUSR the terminal emulator may choose not to make the cursor blink.

AMBIGUOUS_WIDE

Boolean. Indicates if the terminal emulator will render ambiguous-width characters as full-width, taking two cells. Ambiguous width characters are defined by [Unicode Annex 11](#). The width table depends on the unicode version, as reported with the UNICODE_VERSION feature. When true, ambiguous characters take two cells. When false, ambiguous characters take a single cell.

Tests

Examples of ambiguous-width characters that depend on Unicode version:

Code Point	Name	Character	Unicode 8	Unicode 9+
U+0391	GREEK CAPITAL LETTER ALPHA	A	A	A
U+2614	UMBRELLA WITH RAIN DROPS	☂	A	W
U+1F19B	SQUARED THREE D	ⓓ	n/a	A

Note: no new ambiguous-width characters have been added between Unicode 10 and 13.

UNICODE_VERSION

Unsigned integer. Gives the currently supported Unicode version. This determines how the AMBIGUOUS_WIDE feature is interpreted and also suggests how other Unicode features will work, such as which Emoji are present. To advertise a unicode version, conforming terminals must conform to that version of Unicode's behavior as regards:

- Parsing emoji (even if they cannot be displayed).
- Character width, as given by Unicode Annex 11.
- Combining base characters with combining marks.

The values are:

- 0: No unicode support is claimed.
- 8: Unicode version 8 or less.
- N>8: Unicode version N.
- Special encoding-dependent value, if applicable: equal to or larger than maximum encodable version. TODO: Make this more specific when we pick an encoding.

Tests

See AMBIGUOUS_WIDTH.

The following emoji should appear as a single character. If the Unicode version is 8, they must behave like either an East Asian Narrow or Wide character, as specified in TR11. For Unicode 9 and larger, they must behave like any East Asian Wide character. The graphical presentation is up to the terminal emulator.

Unicode Version	Code Points	Name	Character
8+	U+1F1E8 U+1F1F6	flag: Canada	
8+	U+2764 U+FE0F	Red Heart	
9+	U+1F481 U+1F3FD U+200D U+2640 U+FE0F	Woman Tipping Hand: Medium Skin Tone	
10+	U+1F3F4 U+E0063 U+E0061 U+E0062 U+E0063 U+E007F	Flag for British Columbia (CA-BC)	
11+	U+1F9B9 U+1F3FD U+200D U+2640 U+FE0F	Woman Supervillain: Medium Skin Tone	
12+	U+1F468 U+1F3FF U+200D U+1F91D U+200D U+1F468 U+1F3FB	Men Holding Hands: Dark Skin Tone, Light Skin Tone	
13+	U+1F43B U+200D U+2744 U+FE0F	Polar Bear	

TODO: RTL/BIDI

TITLE_SETTING

Boolean. To advertise this feature a terminal emulator should support the following control sequences:

- `CSI 22 ; 0 t`
 - Saves the “icon title” and window title on a stack.
- `CSI 22 ; 1 t`
 - Saves the “icon title” on a stack.
- `CSI 22 ; 2 t`
 - Saves the window title on a stack.
- `CSI 23 ; 0 t`
 - Restores the “icon title” and window title from stack.
- `CSI 23 ; 1 t`
 - Restores the “icon title” from stack.
- `CSI 23 ; 2 t`
 - Restores the window title from stack.
- `OSC 0 ; {title} ST`
 - Sets the window title and “icon title”.
- `OSC 1 ; {title} ST`
 - Sets the “icon title”.
- `OSC 2 ; {title} ST`
 - Sets the window title.

The terminal emulator is free to interpret “icon title” as it sees fit. For example, it could be the tab title. The terminal emulator may choose to use a single stack for both icon and window titles or to use separate stacks for them. The “window title” is an element in the user interface that serves a purpose like a window’s title bar.

esctest

Terminal emulators should pass the following tests to advertise this feature. Note that while title reporting is required for these tests to succeed, terminal emulators are not required to implement title reporting for the sake of this feature.

- `XtermWinopsTests.test_XtermWinops_PushIconAndWindow_PopIconAndWindow`
- `XtermWinopsTests.test_XtermWinops_PushIconAndWindow_PopIcon`

- `XtermWinopsTests.test_XtermWinops_PushIconAndWindow_PopWindow`
- `XtermWinopsTests.test_XtermWinops_PushIcon_PopIcon`
- `XtermWinopsTests.test_XtermWinops_PushWindow_PopWindow`
- `XtermWinopsTests.test_XtermWinops_PushIconThenWindowThenPopBoth`
- `XtermWinopsTests.test_XtermWinops_PushMultiplePopMultiple_Icon`
- `XtermWinopsTests.test_XtermWinops_PushMultiplePopMultiple_Window`
- `XtermWinopsTests.test_XtermWinops_ReportIconLabel`
- `XtermWinopsTests.test_XtermWinops_ReportWindowLabel`

BRACKETED_PASTE

Boolean. Indicates if paste bracketing is supported. Terminal emulators must support:

- `DECSET 2004`
- `DECRST 2004`

When enabled, pasting from the clipboard will inject this code before the pasted text:

- `CSI 2 0 0 ~`

Then the pasted text will be sent. Finally, this code will be injected afterwards:

- `CSI 2 0 1 ~`

When pasting text from the clipboard, terminal emulators should filter out ASCII control codes 0-8, 11, 12, and 14-31 inclusive (that is, everything but Tab, CR, and LF). This ensures that `CSI 201 ~` is not embedded in the pasted contents, and also is a good security practice.

FOCUS_REPORTING

Boolean. Indicates if focus reporting is supported. Terminal emulators must support:

- `DECSET 1004`
- `DECRST 1004`

When enabled, the terminal emulator will send this control sequence when it gains keyboard focus for this session:

- `CSI I`

And when keyboard focus is lost for this session, it will send:

- `CSI O`

CWD

Boolean. Indicates if the terminal emulator can use a working directory report. The terminal emulator must support:

- OSC 7 ; {url} ST

{url} is a file:// URI giving the current directory. It is formatted as:

- file://{hostname}{path}

The {hostname} is the best guess at the current host name (e.g., xyz.example.com) and may be omitted if unknown. The {path} is the absolute path to the current directory and must begin with a slash. This must be a well-formed URI, as described in RFC 3986.

The way in which this information is used by the terminal emulator is implementation defined.

STRIKETHROUGH

Boolean. Indicates if the terminal emulator can render strikethrough text. It must support:

- SGR 9

The terminal emulator may allow strikethrough text to coexist with underlined text.

esctest

This test verifies that SGR 9 is parsed and persisted properly. The terminal emulator must also render it as a line through the text in order to advertise the feature.

- SGRTests.test_SGR_Strikethrough

OVERLINE

Boolean. Indicates if the terminal emulator can render overlined text. It must support:

- SGR 53

escTest

This test verifies that SGR 53 is parsed and persisted properly. The terminal emulator must also render it as a line over the text in order to advertise the feature.

- `SGRTests.test_SGR_Overline`

ITALICS

Boolean. Indicates if the terminal emulator can render italicised text. It must support:

- SGR 3

escTest

This test verifies that SGR 3 is parsed and persisted properly. The terminal emulator must also render it as oblique text or text using an Italic font.

- `SGRTests.test_SGR_Italics`

SYNC

Boolean. Indicates if the terminal emulator supports synchronized updates.

The spec is defined here:

https://gitlab.freedesktop.org/terminal-wg/specifications/-/merge_requests/2

HYPERLINKS

Boolean. Indicates if the terminal emulator supports OSC 8 hyperlinks.

The spec is defined here:

<https://gist.github.com/egmontkob/eb114294efbcd5adb1944c9f3cb5feda>

NOTIFICATIONS

Boolean. Indicates if the terminal emulator supports OSC 9 notifications.

- OSC 9 ; {text} ST

When received, {text} will be displayed to the user outside the normal UI so that even if they are using another application it may still be visible.

SIXEL

Boolean. Indicates if the terminal emulator supports Sixel images.

To conform, this file:

<https://raw.githubusercontent.com/saitoha/libsixel/master/images/snake.six>

Should cause this image to be rendered in the terminal emulator, starting at the cursor's location at the time the file was printed:



CSI_U

Boolean. The “CSI u” method for reporting keystrokes derives from xterm’s `modifyOtherKeys` feature. The original specification for it is here: <http://www.leonerd.org.uk/hacks/fixterms/>

If CSI u is supported but is not currently enabled, this feature should not be advertised. Use `CSI_U_TOGGLEABLE` instead to make the capability known.

Tests

A terminal may advertise this feature if it passes the tests described here:

<https://docs.google.com/document/d/1Is0xKIJTylhuPBVypzJHqNtKsZkStol2WTLfiVSp6RC4/edit>

Some key combinations may be reserved by the platform and cannot be reported. That should not prevent a terminal from advertising the feature if it is generally available and would be useful.

TODO: Script this and get confirmation from Paul that the choices made in the tests are reasonable.

CSI_U_TOGGLEABLE

Boolean. Indicates if CSI_U mode be changed as follows:

- `CSI > 4 ; {param} m`
 - A {param} of 1 enables CSI u mode
 - An absent {param} or a {param} of 0 restores the terminal’s default mode (which may or may not be CSI u).

Publication

Terminals have two mechanisms they may use to publish their feature set:

- An environment variable, `TERM_FEATURES`.
- A control sequence that responds with a report containing the features.

TODO: I am bad at this! Feedback is requested for these control sequences.

The control sequence is:

- CSI = 1 q

The terminal will respond with:

- DCS = | {FeatureString} ST

The environment variable TERM_FEATURES will take the value {FeatureString}.

Encoding

{FeatureString} is a string formed by concatenating an encoding of each feature's value.

A boolean feature is encoded as a short string matching this regular expression:

$$^{[A-Z]}[a-z]^* \$$$

The code must be present if and only if the terminal emulator wishes to advertise the feature. Its absence implies the feature is not advertised.

An integer-valued feature is encoded as a short string matching this regular expression:

$$^{[A-Z]}[a-z]^*[0-9]^+ \$$$

An omitted integer-valued feature implicitly takes the value 0.

For example:

AbCdEf12GH

Features Ab, Cd, Ef, G, and H are present. The value of Ef is 12.

More common features should be assigned shorter codes to conserve space.

In order to give the encoding room to grow in the future, any characters at or after a non-alphanumeric should be ignored. In other words, consider only the prefix of {FeatureString} matching this regular expression:

$$^{[A-Za-z0-9]}^*$$

The codes are:

Type	Name	Code
UInt	24BIT	T

Boolean	CLIPBOARD_WRITABLE	Cw
Boolean	DECSLRM	Lr
UInt	MOUSE	M
Boolean	UNICODE_BASIC	U
Boolean	DECSUSR	Sc
Boolean	AMBIGUOUS_WIDE	Aw
UInt	UNICODE_VERSION	Uv
?	RTL/BIDI	<i>TODO: Egmont</i>
Boolean	TITLE_SETTING	Ts
Boolean	BRACKETED_PASTE	B
Boolean	FOCUS_REPORTING	F
Boolean	CWD	Cw
Boolean	STRIKETHROUGH	Gs
Boolean	OVERLINE	Go
Boolean	ITALICS	Gi
Boolean	SYNC	Sy
Boolean	HYPERLINKS	H
Boolean	NOTIFICATIONS	No
Boolean	SIXEL	Sx
Boolean	CSI_U	U
Boolean	CSI_U_TOGGLEABLE	Ut

The encoded length as specified will be between 0 and 48 bytes (for reasonable values of Uv).

Utilities

We should build small, self-contained files in popular languages to parse {FeatureString}, optionally requesting a report and parsing it in the following languages:

- bash
- C (header-only)
- Go
- Rust
- Python
- Ruby
- Others?

Here is some pseudocode to illustrate what the code will do for a boolean-valued feature. A proper implementation would also include caching and support for integer values.

```
// Read the response from stdin. If it is CSI ... n then return NULL.
// If it is the expected DCS sequence, return its payload and then
// read CSI ... n. If nothing comes back in `timeout` seconds, return
// NULL.
Optional<String> TFReadResponse(Float timeout) {
    // Left as an exercise to the reader
}

Optional<String> TFGetFeatureString(Bool canRequest, Float timeout) {
    let featureString = OS.Environment["TERM_FEATURES"]
    if featureString != NULL { return featureString }
    if not canRequest { return NULL }

    // Write a request for FeatureString report and DSR.
    if Posix.write(0, "\e[=1q\e[5n") < 0 { return NULL }
    return TFReadResponse(timeout)
}

// Converts a feature string like "AbCd12" into a map like
// { Ab: 1, Cd: 12 }
Optional<Map<String, Int>> TFParse(Optional<String> featureString) {
    // Left as an exercise to the reader
}

Int TFGetValue(Optional<String> featureString, String name) {
    let map = TFParse(featureString)
    if name not in map { return False }
    return map[name]
}
```

We should build a command-line tool that queries the terminal and sets `TERM_FEATURES` that people can put in their `.profile`.

We should also build a command-line tool to help users inspect the available capabilities.